

ML ACADEMY

Deep Learning Interview Handbook

Detailed Questions, Explanations & Worked Examples for Top Tech Interviews

Neural Networks • Backpropagation • CNNs • RNNs • Attention • Transformers Training •
Optimisation • Regularisation • Vision • Sequence Models • Production

www.learnmlacademy.com

Free Resource - Share with your network

How to get the most from this handbook

This PDF is designed for serious interview preparation. Each topic combines an interview-ready answer with deeper explanation, examples, diagrams, code, calculations or system trade-offs wherever they improve understanding.

Recommended approach: answer the question yourself first, then compare with the explanation and work through the example.

For coding and SQL questions, state assumptions, approach, complexity and edge cases before writing code. For ML/AI system design, discuss requirements, trade-offs, failure modes, monitoring and security instead of claiming one architecture is always best.

Section	Topics
1. Research basis and further reading	1
TOTAL COVERAGE	1

1. Gradients, normalization and training foundations

Start with mechanisms. If you can derive a tiny example and explain the failure mode, most follow-ups become easier.

What modern deep-learning interviews actually test Expect the interviewer to move between mathematics, tensor shapes, training behavior and production constraints. A strong answer names the mechanism, gives a small example, and explains what evidence would confirm the diagnosis.

INTERVIEW QUESTION

Walk through backpropagation on a one-weight network.

What the interviewer is testing Whether you can connect a forward pass, loss, local derivatives, chain rule and optimizer update instead of memorizing the word backpropagation.

Interview-ready answer

Backpropagation computes how much each parameter contributed to the loss by applying the chain rule from the loss back toward the inputs. The gradient tells the optimizer which direction changes the loss locally.

Detailed explanation

Take $y_{\text{hat}} = w \cdot x + b$ and squared error $L = (y - y_{\text{hat}})^2$. First run the forward pass. Then differentiate L with respect to y_{hat} , multiply by the derivative of y_{hat} with respect to w , and update w in the opposite direction of the gradient. In a large neural network, automatic differentiation performs the same local-chain-rule bookkeeping across a computation graph.

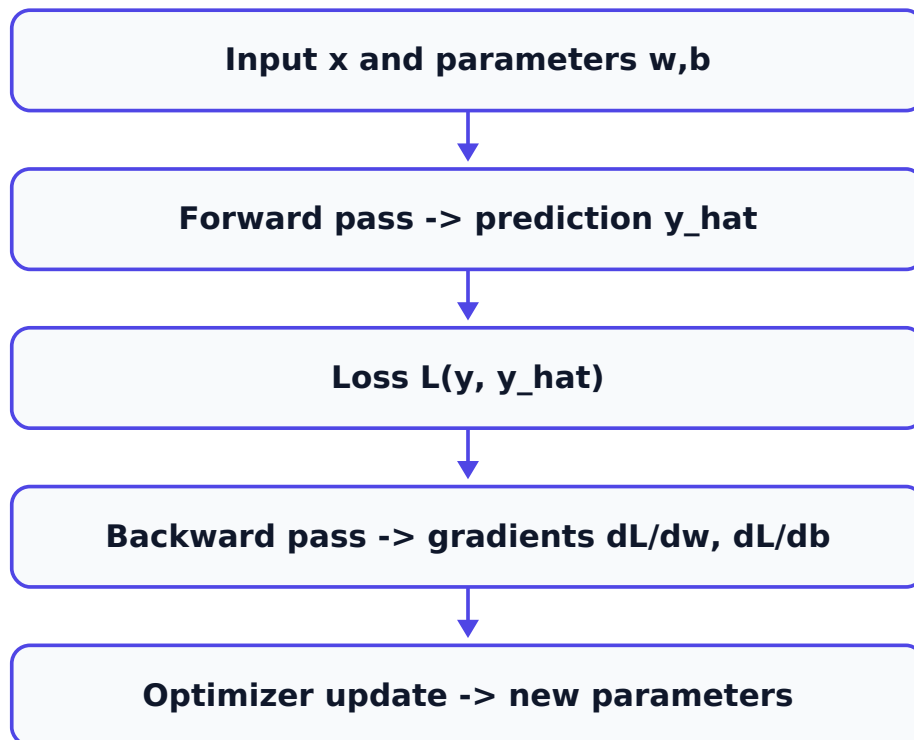


Figure: conceptual flow for this answer.

Example

Let $x=2$, $w=3$, $b=1$ and $y=10$. Forward: $y_{\text{hat}}=7$ and $L=(10-7)^2=9$. $dL/dy_{\text{hat}}=2*(7-10)=-6$. $dy_{\text{hat}}/dw=x=2$. Therefore $dL/dw=-12$. With learning rate 0.01, $w_{\text{new}} = 3 - 0.01*(-12) = 3.12$. The weight rises because the prediction was too small.

```
import torch

x = torch.tensor([2.0])
y = torch.tensor([10.0])
w = torch.tensor([3.0], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)

y_hat = w * x + b
loss = (y - y_hat).pow(2).mean()
loss.backward()

print("prediction:", y_hat.item()) # 7.0
print("loss:", loss.item()) # 9.0
print("dL/dw:", w.grad.item()) # -12.0
```

Likely follow-up questions

- Why is the gradient negative in this example?
- What changes if the activation function is ReLU?
- Why do we normally average loss across a mini-batch?

Weak answer to avoid

Saying only 'backprop uses the chain rule' without showing what is differentiated or how the gradient becomes a parameter update.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION2026 SIGNAL

Training-stability questions remain common because they expose whether the candidate can diagnose optimization rather than just name architectures.

Why do vanishing and exploding gradients occur?

Interview-ready answer

Repeated multiplication by Jacobians across many layers or time steps can shrink gradients toward zero or grow them extremely large. This makes early layers learn too slowly or destabilizes optimization.

Detailed explanation

If typical derivative magnitudes are below 1, multiplying many of them can make the gradient exponentially small; if they are consistently above 1, gradients can grow. Saturating activations, poorly scaled initialization and long recurrent chains make the problem worse. Residual connections, suitable initialization, normalization, gated recurrent units, careful activations and gradient clipping are common mitigations. Clipping controls the symptom of large gradients but does not fix a fundamentally unstable model.

Example

A derivative magnitude of 0.5 repeated over 20 transformations gives 0.5^{20} , about 0.00000095. A derivative magnitude of 1.5 repeated 20 times gives more than 3,000.

Likely follow-up questions

- Why do residual connections help gradient flow?
- What would you log to detect exploding gradients?
- Why did LSTMs help compared with vanilla RNNs?

Weak answer to avoid

Treating gradient clipping as the complete solution to every exploding-gradient problem.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION**What is the difference between Batch Normalization and Layer Normalization?****Interview-ready answer**

BatchNorm normalizes feature activations using statistics aggregated across a mini-batch; LayerNorm normalizes across features within each individual example. LayerNorm does not depend on other examples in the batch and is therefore natural for Transformers and variable-length sequence settings.

Detailed explanation

During training, BatchNorm estimates batch mean and variance and typically maintains running statistics for inference. Very small or non-IID batches can make those estimates noisy. LayerNorm uses the current example's hidden features, so training and inference behavior are more consistent. The exact axes depend on tensor layout, but the interview-level distinction is batch-dependent population statistics versus per-example feature normalization.

Example

For an image tensor $[N, C, H, W]$, BatchNorm2d normalizes each channel using statistics over N, H, W . For a Transformer hidden vector of dimension d , LayerNorm normalizes the d values for each token independently.

```
import torch.nn as nn

bn = nn.BatchNorm2d(64)    # typical CNN feature map
ln = nn.LayerNorm(768)     # typical Transformer hidden state
```

Likely follow-up questions

- Why can BatchNorm be awkward with batch size 1?
- Where is LayerNorm placed in pre-norm versus post-norm Transformers?

Weak answer to avoid

Saying BatchNorm is always better for CNNs and LayerNorm always better for everything else; the architecture and deployment constraints matter.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

2. Optimization and efficient training

Interviewers often turn a theoretical question into an engineering one: memory, stability, learning rate, batch size and evidence from training curves.

INTERVIEW QUESTION

SGD with momentum or Adam: how do you choose?

Interview-ready answer

Adam adapts the step size per parameter using moving estimates of first and second moments and usually reaches a useful solution quickly. SGD with momentum is simpler and can generalize very well when learning-rate schedules and large datasets are tuned carefully.

Detailed explanation

There is no universal winner. Adam-like optimizers are strong defaults for Transformers and noisy or sparse gradients. SGD with momentum remains common in vision and large supervised training where a carefully tuned schedule can yield excellent final quality. In an interview, compare convergence speed, sensitivity to learning rate, memory overhead, generalization, batch size and the amount of tuning the team can afford.

Example

If a research team needs a stable baseline tomorrow, AdamW may be the pragmatic starting point. If the final production vision model will be trained repeatedly on a massive dataset, an SGD schedule may be worth the extra tuning if it improves the final validation metric.

Likely follow-up questions

- What does weight decay do in AdamW?
- Why can very large batches require a learning-rate adjustment?
- What is momentum intuitively?

Weak answer to avoid

Saying Adam is always better because it is newer.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION2026 SIGNAL

Modern MLE interviews increasingly connect model training to GPU memory, throughput and deployment economics.

What does mixed-precision training change?

Interview-ready answer

Mixed precision performs many operations in FP16 or BF16 to reduce memory use and increase accelerator throughput while retaining higher precision where numerical range or accumulation accuracy matters.

Detailed explanation

Lower precision cuts activation and tensor bandwidth and may enable a larger batch. FP16 has a relatively narrow exponent range, so small gradients can underflow; loss scaling is often used. BF16 has the exponent range of FP32 with fewer mantissa bits and is generally more forgiving. Optimizer states and master weights may still use higher precision, so end-to-end memory does not always fall by exactly half.

Example

A model that uses 18 GB for parameters and activations in FP32 may fit a significantly larger batch under BF16, but Adam optimizer states can still be a large memory consumer.

```
# PyTorch autocast pattern
with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
    logits = model(x)
    loss = criterion(logits, y)

loss.backward()
optimizer.step()
optimizer.zero_grad(set_to_none=True)
```

Likely follow-up questions

- Why is BF16 often easier to train than FP16?
- What operations might remain FP32?
- How would you validate that mixed precision did not change model quality?

Weak answer to avoid

Assuming every tensor and optimizer state becomes half the size.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION

What is gradient clipping, and when is it useful?

Interview-ready answer

Gradient clipping limits the gradient magnitude before the optimizer step, commonly by scaling the full gradient vector when its norm exceeds a threshold.

Detailed explanation

It is especially useful when rare batches or long recurrent/sequence computations produce unusually large gradients. Global-norm clipping preserves the gradient direction while reducing magnitude. If clipping activates on nearly every step, investigate learning rate, initialization, normalization, loss scaling or corrupted data rather than hiding the underlying problem.

Example

If the total gradient norm is 20 and `max_norm=1`, all gradients are scaled by roughly $1/20$ before the optimizer uses them.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

Likely follow-up questions

- What would you log around clipping?
- How is clipping by value different from clipping by norm?

Weak answer to avoid

Treating clipping as a substitute for diagnosing an unstable training setup.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION2026 SIGNAL

Loss-design questions matter in vision and imbalanced deep-learning roles where class imbalance happens inside optimization rather than only at the final threshold.

What is focal loss and why was it introduced?

Interview-ready answer

Focal loss modifies cross-entropy so that easy, already-correct examples contribute less and hard examples receive relatively more training weight.

Detailed explanation

For binary classification, the focal factor is often written $(1-p_t)^\gamma$ multiplied by cross-entropy. When p_t is already close to 1, the factor becomes small. This was useful in dense object detection where the model sees enormous numbers of easy background anchors. It is one tool for imbalance; class weighting, sampling and threshold design may be more appropriate in other problems.

Example

With $\gamma=2$, an easy example with $p_t=0.95$ receives a factor $(0.05)^2=0.0025$, while a hard example with $p_t=0.6$ receives 0.16. The hard example therefore contributes far more relative weight.

Likely follow-up questions

- What happens when $\gamma=0$?
- How would you compare focal loss against class-weighted cross-entropy?
- Can focal loss hurt calibration?

Weak answer to avoid

Using focal loss automatically whenever the positive class is rare.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

3. CNNs, sequence models and Transformers

Architecture questions should connect the computation to the inductive bias and the production consequence.

INTERVIEW QUESTION

How does a convolutional layer work, and what is a receptive field?

Interview-ready answer

A convolutional layer slides learnable kernels over local spatial neighborhoods. Weight sharing lets the same detector respond at many locations. The receptive field is the region of the original input that can influence one output activation.

Detailed explanation

Kernel size, stride, dilation and depth determine how spatial information is aggregated. Stacking 3x3 convolutions grows the effective receptive field while adding nonlinearities. Pooling or strided convolution reduces resolution. CNNs encode locality and translation-related inductive bias, which is why they remain strong for many visual tasks even in the Transformer era.

Example

For a 32x32 image, a 3x3 convolution with padding=1 and stride=1 keeps 32x32 resolution. A following stride-2 layer reduces it to roughly 16x16 while each output sees a wider region of the input.

```
import torch.nn as nn
conv = nn.Conv2d(in_channels=3, out_channels=64,
                  kernel_size=3, stride=1, padding=1)
```

Likely follow-up questions

- How do you calculate output shape?
- What does dilation change?
- Why do 1x1 convolutions matter?

Weak answer to avoid

Describing convolution only as 'feature extraction' without explaining local connectivity and weight sharing.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION

Why do LSTMs and GRUs help compared with a vanilla RNN?

Interview-ready answer

They introduce gated paths that control what information is written, retained and exposed, making it easier for gradients and useful state to persist across longer sequences.

Detailed explanation

A vanilla RNN repeatedly transforms hidden state, so long chains can suffer severe vanishing or exploding gradients. LSTM uses input, forget and output gates around a cell state; GRU uses a simpler update/reset structure. They are still sequential and therefore harder to parallelize than Transformers, but remain useful for some streaming and small-data sequence tasks.

Example

For a sensor stream where each prediction arrives one step at a time and the sequence is not extremely long, a compact GRU can be cheaper than a large Transformer.

Likely follow-up questions

- Why are Transformers more parallel during training?

- What is teacher forcing in sequence models?
- When would you still choose an RNN?

Weak answer to avoid

Saying LSTMs completely solve long-term dependencies.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION2026 SIGNAL

Recent AI-engineer interviews explicitly probe self-attention mechanics and scaling.

Explain scaled dot-product self-attention step by step.

Interview-ready answer

Each token is projected into a query, key and value. Query-key dot products produce relevance scores; divide by $\sqrt{d_k}$, apply any mask, softmax into weights, then use those weights to form a weighted sum of values.

Detailed explanation

The matrix form is $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$. Scaling prevents dot products from growing too large as the key dimension increases, which would push softmax into saturated regions. Multi-head attention repeats this in multiple learned subspaces so different relationships can be modeled simultaneously.

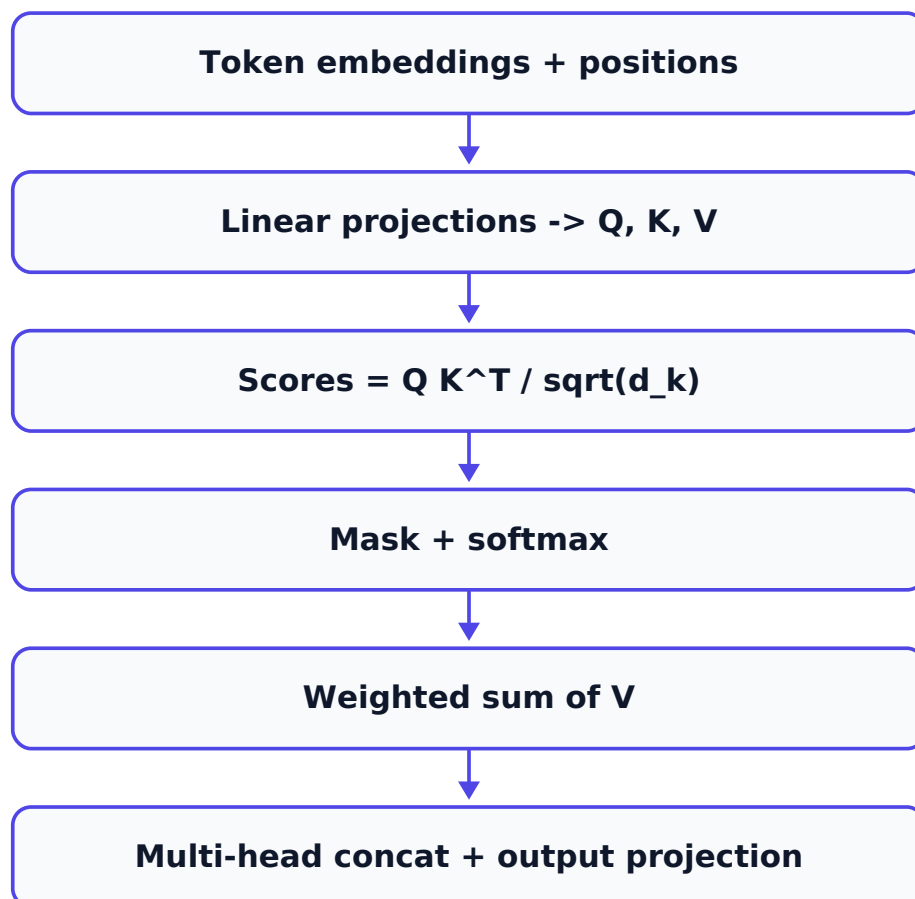


Figure: conceptual flow for this answer.

Example

If one token's query is very similar to a previous token's key, their dot product is high, its softmax weight grows, and that previous token's value contributes strongly to the new representation.

```
# conceptual PyTorch call
out = torch.nn.functional.scaled_dot_product_attention(
    query, key, value,
    attn_mask=mask,
    dropout_p=0.0,
    is_causal=True,
)
```

Likely follow-up questions

- Why do we divide by $\sqrt{d_k}$?
- What changes for causal attention?
- What does multi-head attention buy us?

Weak answer to avoid

Calling attention a database lookup. It is a differentiable weighted aggregation over representations.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION 2026 SIGNAL

Self-attention complexity and efficient-attention follow-ups are increasingly common in Transformer-focused interviews.

Why is dense self-attention $O(n^2)$ in sequence length?

Interview-ready answer

A dense attention layer forms interactions between every query token and every key token, creating an n -by- n score matrix. Doubling sequence length therefore roughly quadruples the number of pairwise scores.

Detailed explanation

The total layer cost also includes projections and hidden dimension, but the n^2 attention matrix becomes important for long contexts. Efficient kernels such as FlashAttention reduce memory traffic and avoid materializing large intermediates, while sparse or structured attention methods can alter the computation pattern itself.

Example

A 4,096-token sequence has about 16.8 million token pairs per head. At 8,192 tokens, that rises to about 67.1 million - roughly 4x.

Likely follow-up questions

- What does FlashAttention optimize?
- Does FlashAttention change the mathematical result?
- Name strategies that actually change long-context complexity.

Weak answer to avoid

Saying attention is $O(n)$ because GPU operations run in parallel.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

4. Serving, debugging and transfer learning

A strong deep-learning candidate can explain not only why the network learns, but also how it fails and how it is served under memory and latency constraints.

INTERVIEW QUESTION2026 SIGNAL

Hugging Face now documents multiple KV-cache strategies because memory/latency trade-offs are central to LLM serving.

What is a KV cache and why does it speed autoregressive inference?

Interview-ready answer

During autoregressive generation, keys and values for previous tokens do not change. A KV cache stores them so the model does not recompute those projections at every new token.

Detailed explanation

At decoding step t , the new query attends to cached keys and values from previous positions plus the current token's new key/value. This saves repeated computation but consumes memory that grows with active sequence length, layers, heads and concurrent requests. Cache policy therefore becomes a throughput and memory design decision.

Example

Without a cache, producing token 501 can repeat key/value work for the previous 500 tokens. With a cache, those tensors are reused. For long contexts and many users, however, the cached state itself can become the GPU-memory bottleneck.

```
# Hugging Face generation uses a cache by default for many causal LMs
outputs = model.generate(
    **inputs,
    max_new_tokens=100,
    use_cache=True,
)
```

Likely follow-up questions

- What is a static versus dynamic cache?
- How does long context affect KV-cache memory?
- Why can quantized or offloaded cache help?

Weak answer to avoid

Confusing a KV cache with conversational memory or a vector database.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION

How would you debug a neural network whose loss does not decrease?

Interview-ready answer

Start with the simplest checks: validate labels and input ranges, overfit a tiny batch, inspect the loss/activation choice, verify gradients exist and are finite, then test learning rate, initialization and normalization.

Detailed explanation

The tiny-batch test is powerful: if a high-capacity model cannot drive loss very low on 10-50 examples, the pipeline or optimization is probably broken. Log gradient norms, activation statistics, class distribution and parameter updates. Check that training mode is enabled, frozen parameters are intentional, and preprocessing is identical to what the model expects.

Example

If training accuracy stays at random chance and every gradient norm is zero, inspect accidental detach/no_grad, frozen parameters or saturating activations before tuning the architecture.

```
# quick gradient sanity check
loss.backward()
for name, p in model.named_parameters():
    if p.requires_grad:
        if p.grad is None:
            print(name, "NO GRAD")
        else:
            print(name, float(p.grad.norm()))
```

Likely follow-up questions

- What does it mean if loss is NaN?
- Why overfit one mini-batch?
- How do you distinguish data bugs from learning-rate issues?

Weak answer to avoid

Immediately adding layers or training longer without collecting evidence.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION2026 SIGNAL

Current ML system-design preparation increasingly includes GPU batching and LLM-serving trade-offs.

How do continuous batching and batch size affect inference latency and throughput?

Interview-ready answer

Batching increases accelerator utilization by processing multiple requests together, but waiting to form large batches can increase per-request latency. Continuous batching admits new decoding work as other requests finish so the GPU stays busy.

Detailed explanation

Interactive serving optimizes multiple metrics: time to first token, per-token throughput, p95 completion latency, fairness and memory. Long sequences can monopolize KV cache and compute. A scheduler may bucket compatible requests, enforce token budgets and use admission control under overload.

Example

A static batch of eight requests can waste capacity if two finish quickly while six continue. Continuous batching can fill freed slots with new requests instead of waiting for the whole original batch.

Likely follow-up questions

- Why can maximizing GPU utilization harm p95 latency?
- How would you handle one extremely long prompt?
- What is admission control?

Weak answer to avoid

Choosing the largest possible batch size without a latency SLO.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

INTERVIEW QUESTION

When should you use transfer learning, freeze layers, or fine-tune end to end?

Interview-ready answer

Start from a pretrained model when its learned representation overlaps your task and your labeled dataset is limited. Freeze more layers when data or compute is scarce; progressively unfreeze when the target domain differs and you have enough evidence to justify adapting the representation.

Detailed explanation

Early layers often learn reusable low-level patterns, while later layers are more task-specific. Full fine-tuning offers maximum flexibility but costs more memory and increases overfitting risk. Parameter-efficient methods can be a middle ground for very large models.

Example

For a medical-image classifier with 5,000 labeled images, start from a pretrained vision backbone, train a new classification head, then unfreeze later blocks with a smaller learning rate while monitoring validation performance.

Likely follow-up questions

- What happens if the source and target domains are very different?
- Why use a smaller learning rate for pretrained layers?
- How would you detect catastrophic forgetting?

Weak answer to avoid

Assuming pretrained weights always help regardless of domain mismatch.

Best interview habit

Answer the main question first, then volunteer one trade-off and pause for the interviewer.

Deep-dive deep learning tutorial: gradients to serving

This section connects the mathematics of neural networks to architecture, optimization, memory and production inference.

Study method Do not memorize definitions only. For each concept, explain the idea, work the example, name a failure mode, and state one trade-off. That pattern matches how stronger technical interviews are evaluated.

1. One neuron, forward pass and backpropagation

A neuron computes $z = w \cdot x + b$ and passes z through an activation. Training adjusts parameters to reduce a loss. Backpropagation is reverse-mode automatic differentiation: apply the chain rule from the loss backward through each operation so every parameter receives a gradient.

The important interview skill is to trace shapes and derivatives, not simply say “the chain rule”.

Worked example / interview scenario

$x=2$, $w=3$, $b=1$ gives $y_{\text{hat}}=7$ for a linear neuron. If target $y=10$ and $L=(y_{\text{hat}}-y)^2$, $dL/dy_{\text{hat}}=2(7-10)=-6$ and $dy_{\text{hat}}/dw=x=2$, so $dL/dw=-12$. With learning rate 0.01, w becomes 3.12.

2. Activation functions and saturation

ReLU is cheap and avoids positive-side saturation but can produce dead neurons when units stay negative. Sigmoid and tanh saturate for large magnitudes, shrinking gradients. GELU is common in Transformers because it is smooth and gates values probabilistically.

Activation choice interacts with initialization and normalization; do not discuss it in isolation.

Activation	Strength	Common concern
ReLU	simple, sparse positive activations	dead units
tanh	zero-centered	saturation
sigmoid	probability-like output	saturation; not zero-centered
GELU	smooth, Transformer-friendly	slightly more compute

3. Initialization and signal propagation

If weights are too large, activations and gradients can explode; too small and signals vanish. Xavier/Glorot initialization targets variance preservation for roughly symmetric activations, while He initialization is suited to ReLU-like activations.

Initialization matters most at the start of optimization; normalization and residual connections further stabilize deep networks.

Worked example / interview scenario

For a ReLU layer with $\text{fan_in}=512$, He initialization uses a variance around $2/512$. The exact random draw differs, but the objective is stable activation scale across layers.

4. BatchNorm, LayerNorm and RMS-style normalization

BatchNorm normalizes activations using batch statistics and maintains running statistics for inference. It works especially well in many CNNs but depends on batch behavior. LayerNorm normalizes within each example across features and is natural for sequence models and Transformers.

Normalization improves optimization but is not identical to regularization. Small or non-IID batches can make BatchNorm difficult.

Worked example / interview scenario

A Transformer processes variable-length sequences and often uses LayerNorm/RMS-style normalization because behavior should not depend on other examples in the batch.

5. Optimizers: SGD, momentum and Adam

SGD follows the minibatch gradient. Momentum accumulates a velocity that smooths noisy directions. Adam adapts step sizes using moving averages of first and second moments and is convenient for deep networks.

Generalization, learning-rate schedules and weight decay matter as much as optimizer brand. AdamW decouples weight decay from the adaptive update.

Worked example / interview scenario

A common workflow is AdamW with warmup and cosine/linear decay for Transformers; for vision, carefully tuned SGD with momentum can still be competitive.

Deep dive continued

6. Regularization: weight decay, dropout, augmentation and early stopping

Regularization reduces overfitting by constraining the effective hypothesis space or injecting useful noise. Weight decay discourages large weights. Dropout randomly zeroes activations during training. Data augmentation expands the observed variability. Early stopping prevents over-optimization on the training set.

Choose methods based on data and architecture; stacking every regularizer can underfit.

Worked example / interview scenario

For image classification with limited data, transfer learning plus realistic image augmentation may help more than simply increasing dropout.

7. Convolution and receptive field

A convolution applies learned kernels across local neighborhoods with shared weights, exploiting spatial locality. Stride controls movement; padding controls boundary size; dilation expands receptive field without increasing kernel size.

The receptive field of deeper units grows as layers compose, allowing local edges to become shapes and objects.

Worked example / interview scenario

A 3x3 kernel over an RGB image has 3x3x3 weights per output channel (plus optional bias). Sharing those weights across positions makes convolution parameter-efficient compared with a fully connected layer on pixels.

8. RNN, LSTM and GRU intuition

A vanilla RNN repeatedly transforms the current input and previous hidden state. Long dependency chains can make gradients vanish or explode. LSTM and GRU introduce gates that control information flow and create shorter gradient paths through state.

Transformers displaced many RNNs for parallelizable sequence modeling, but recurrent models can still be useful in streaming or constrained settings.

Worked example / interview scenario

For sensor data arriving one step at a time on an edge device, a compact GRU may be simpler than a large Transformer.

9. Self-attention and multi-head attention

Scaled dot-product attention computes $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$. Multi-head attention uses several learned projections so different heads can model different relationships. Causal masking prevents future-token leakage in autoregressive decoders.

Dense self-attention has quadratic score-matrix size in sequence length, motivating sliding-window, sparse or linear alternatives for long contexts.

Worked example / interview scenario

With sequence length $n=4$, each head forms a 4x4 attention-score matrix. Doubling n to 8 quadruples the number of pairwise scores.

10. Residual connections and Transformer blocks

Residual connections add a block input to its transformed output, creating direct gradient paths that enable very deep networks. A Transformer block combines attention, an MLP/feed-forward network, normalization and residual connections.

Pre-norm versus post-norm placement changes training stability and depth behavior; the interview point is to understand the role of each component.

Deep dive continued

11. Mixed precision, gradient scaling and memory

Mixed-precision training uses lower-precision formats for speed and memory efficiency while retaining higher precision where needed. FP16 can underflow small gradients, so gradient scaling multiplies the loss before backward and rescales gradients afterward. BF16 has a larger exponent range and often needs less scaling.

Memory also comes from activations, optimizer states and gradients, not only weights. Gradient checkpointing trades extra compute for lower activation memory.

Worked example / interview scenario

If a model fits parameters but OOMs during training, inspect activation and optimizer-state memory; quantizing weights alone may not solve training memory.

12. Inference: quantization, KV cache and batching

Inference optimization differs from training. Quantization reduces memory bandwidth and model footprint. KV caching reuses past attention keys/values during autoregressive decoding. Continuous batching keeps accelerators busy by admitting new sequences as others finish.

Track time-to-first-token, time-per-output-token, throughput and p95 latency. Optimizing only average latency can hide poor tail performance.

Complete case walkthrough

Case study - transfer learning for visual defect detection

Problem: fine-tune an image classifier for a small industrial defect dataset.



Explain why transfer learning reduces data requirements, how augmentation should reflect plausible visual variation, why class imbalance may make recall/PR-AUC more useful than accuracy, and how you would examine Grad-CAM or error clusters without claiming causal explanations.

Additional interview drill

Answer each question aloud in 60-90 seconds. Then add one example, one failure mode and one trade-off.

#	Question	Strong answer should mention
1	Derive dL/dw for one linear neuron.	chain rule with actual numbers
2	Why does ReLU help vanishing gradients?	non-saturating positive region
3	BatchNorm vs LayerNorm?	normalization axes, batch dependence, use cases
4	Why residual connections?	short gradient path and easier deep optimization
5	Why scale QK^T by $\sqrt{d_k}$?	stabilize logits/softmax as dimension grows
6	Why $O(n^2)$ attention?	all pairwise token scores
7	What does mixed precision change?	speed/memory, numerical stability, scaling
8	What is gradient clipping?	bound gradient norm/value, useful for exploding gradients
9	What is gradient checkpointing?	recompute activations to save memory
10	When freeze layers?	small data, transfer, faster/stable adaptation
11	What is KV cache?	reuse past K/V at inference, memory trade-off
12	How diagnose flat loss?	data/labels, LR, gradients, initialization, activation, bugs

Hands-on labs and worked exercises

These exercises turn the concepts into things you can calculate, code, debug or design. Work them before reading the expected reasoning.

Lab 1 - Backpropagate through a two-operation graph

Goal Compute gradients instead of memorizing “chain rule”.

Steps

1. Let $z=wx+b$, $y_{\text{hat}}=z^2$, and $L=(y_{\text{hat}}-y)^2$.
2. Use $x=2, w=1, b=1, y=16$.
3. Compute z , y_{hat} , dL/dy_{hat} , dy_{hat}/dz , dz/dw and dL/dw .

Expected reasoning / result

$z=3$, $y_{\text{hat}}=9$. $dL/dy_{\text{hat}}=-14$, $dy_{\text{hat}}/dz=6$, $dz/dw=2$, so $dL/dw=-168$. Large gradients motivate sensible learning rates/clipping in deeper systems.

Interview connection

Narrate each local derivative and multiply along the path.

Lab 2 - Convolution output shape

Goal Reason about kernel, stride and padding.

Steps

1. Input is 32x32, kernel 3, stride 1, padding 1. Compute output size.
2. Repeat with stride 2.
3. Explain why padding 1 preserves size only for stride 1 in this case.

Expected reasoning / result

Formula $\text{floor}((N+2P-K)/S)+1$. Results: 32 for stride 1, 16 for stride 2.

Interview connection

Shape reasoning is a common CNN follow-up.

Lab 3 - Attention matrix shapes

GoalTrack Q, K, V dimensions.

Steps

1. Batch=2, heads=8, sequence=128, head_dim=64.
2. State shapes of Q, K, V and QK^T .
3. Estimate number of attention-score values per batch.

Expected reasoning / result

Q/K/V: [2,8,128,64]; scores: [2,8,128,128] = 262,144 values. Doubling sequence length quadruples score values.

Interview connection

This makes the quadratic sequence-length cost concrete.

Lab 4 - Diagnose flat training loss

Goal Use a structured debugging sequence.

Steps

1. Overfit a tiny batch. If it cannot overfit, suspect implementation/data rather than generalization.
2. Check label alignment and loss target format.
3. Inspect gradient norms and parameter updates.
4. Try a known-safe learning rate and initialization.
5. Disable augmentation/regularization temporarily.

Expected reasoning / result

The “overfit a tiny batch” test is powerful because a sufficiently expressive network should memorize a handful of examples.

Interview connection

Debug from simplest falsifiable hypotheses rather than changing five hyperparameters at once.

Lab 5 - Estimate inference bottleneck

Goal Separate prefill from decode.

Steps

1. Compare request A: 12k-token prompt + 50 output tokens and B: 200-token prompt + 1500 output tokens.
2. Predict which has higher TTFT and which spends longer decoding.
3. Explain how KV cache and continuous batching help.

Expected reasoning / result

A tends to have heavier prefill/TTFT; B has longer serial decode. KV cache avoids recomputing old K/V, while continuous batching improves accelerator utilization.

Interview connection

Serving questions reward workload-aware reasoning, not just naming optimizations.

Last-minute revision sheet

Use this page in the final 30-60 minutes before an interview. Say each answer aloud in one or two sentences; if you cannot, revisit the relevant chapter.

Topic	One-line recall
Backpropagation	Forward pass -> loss -> chain-rule gradients -> optimizer update.
Gradient problems	Repeated Jacobian multiplication can shrink or explode gradients; diagnose before clipping.
BatchNorm vs LayerNorm	Batch statistics across examples vs per-example normalization across hidden features.
Adam vs SGD	Adam is a strong fast-convergence default; SGD can be excellent with tuning and schedules.
Mixed precision	Lower-precision compute saves memory/throughput; validate numerical stability and final quality.
Self-attention	$\text{softmax}(\mathbf{QK}^T/\sqrt{d_k})\mathbf{V}$; dense attention forms an $n \times n$ score matrix.
KV cache	Reuse past K/V during decoding; faster generation but more memory.
Debugging	Overfit a tiny batch, check labels/loss/gradients/learning rate before redesigning the model.

Research basis and further reading

This handbook is original LearnMLAcademy material. Public interview reports and official technical documentation were used to identify current topic emphasis and production practices. The explanations, examples, diagrams and code in this handbook are independently written.

Aced / Exponent - Top Deep Learning Interview Questions (2026)

<https://www.tryexponent.com/blog/top-deep-learning-interview-questions>

Current deep-learning question bank reviewed by experienced ML engineers.

PyTorch - Scaled Dot Product Attention

https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention

Official implementation notes and optimized attention backends including FlashAttention-2 and memory-efficient attention.

Hugging Face Transformers - KV Cache Strategies

https://huggingface.co/docs/transformers/kv_cache

Current cache types, trade-offs, offloading, quantized cache and iterative generation.

Aced / Exponent - Meta Senior MLE Interview Experience (2026)

<https://www.tryexponent.com/experiences/meta-facebook-senior-machine-learning-engineer-interview-8719a7>

Candidate-reported Meta loop covering classic coding, AI-assisted debugging, behavioral questions and recommendation-system design with online/offline evaluation.

Google Machine Learning Crash Course

Official ML learning material emphasizes practical exercises, production systems, testing and monitoring in addition to model theory.

Made With ML MLOps Course

Production-oriented curriculum covering data, modeling, testing, reproducibility, serving, CI/CD and monitoring.

Current 2026 interview-preparation resources

Public guides increasingly emphasize production trade-offs, system design, debugging and role-specific reasoning rather than definition-only question banks.

Keep learning at learnmlacademy.com

167 in-depth lessons • Python code examples • Visual diagrams • Projects • Interview preparation • Generative AI • LLM & RAG • Agentic AI • Always free